Data Visualization

Engineering Computation with Python — Chapter 7

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. Specialized Plots
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. Summary

© Muním Zabidi

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. Specialized Plots
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. Summary

A Formula Tells You What. A Graph Tells You How.

Data visualization

Turning numbers into pictures that can be reasoned about — this is where engineering mathematics becomes visible, and among the most practical skills in this course.

- A graph summarizes huge amounts of information into one picture.
- A graph makes it easy to compare multiple sets of measured data.
- A graph shows directly how one quantity changes in response to another.

© Munim Zabidi

Every Measurement Has Two Vectors

Recording scientific data always involves at least two vectors:

Measured variable	Function variable
The quantity being observed (plotted on the y-axis).	The quantity the measurement depends on — time, angle, distance (plotted on the x-axis).

Example: $V(t)$ is a voltage *reading* (measured variable) over *time* (function variable). Both vectors must have the same length.

© Muním Zabidi

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. Specialized Plots
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. Summary

The plot Function

`matplotlib.pyplot` makes Matplotlib work like MATLAB. Each `pyplot` function changes a figure in some way:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
plt.show()
```

This is the simplest possible plot — one array, default styling, no axis labels on the x-axis.

Example: Plotting a Function

Plot $y = e^{x/2}$ for $x = 1, \ldots, 10$:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(1, 11)
y = np.exp(x / 2)
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('exp(x/2)')
plt.title('Exponential Plot')
plt.show()
```

Result: a smooth curve rising steeply — $e^{x/2}$ grows faster and faster as x increases.

Line and Marker Styles

A compact format string combines color, marker, and line style — e.g. `'-mo'` for a solid magenta line with circle markers.

Spec	Style
`'-'`	solid
`':'`	dotted
`'--'`	dashed
`'-.'`	dash-dot

Common markers: `'.'` point, `'o'` circle, `'s'` square, `'^'` triangle, `'*'` star.

Colors: `b`, `g`, `r`, `c`, `m`, `y`, `k`, `w`.

© Muním Zabidi

Example: A Styled Sine Wave

```
x = np.arange(1, 501, 10)
y = np.sin(5 * x)
plt.plot(x, y, '-mo')   # magenta line, circle markers
plt.xlabel('x')
plt.ylabel('sin(5x)')
plt.title('Sine Wave Plot')
plt.show()
```

Result: an oscillating curve, each sample point marked with a magenta circle, connected by a solid magenta line.

Table of Contents

1 Why Visualize Data?

2 2-D Line Plots

3 Plot Formatting

4 Subplots

5 Specialized Plots

6 Statistical Plots

7 The Bench Experiment

8 Preview: Slope

9 Summary

Decorating a Plot

Properties	Function
Title, axis labels, legend	`title`, `xlabel`, `ylabel`, `legend`
Axis limits	`xlim`, `ylim`, `axis`
Grid and ticks	`grid`, `xticks`, `yticks`
Multiple plots	`figure`, `subplot`

A plot is not complete just because it displays

It needs axis labels with units, a title, and a legend when more than one curve is shown.

© Muním Zabidi

LaTeX Math Symbols in Plot Labels

Matplotlib renders LaTeX-style math inside a raw string between $ symbols:

```
plt.xlabel(r'$\omega$')
plt.title(r'Sin($\omega$) Graph')
```

Note the leading r

`r'...'` is a raw string, needed so Python does not try to interpret the backslash. This is matplotlib's own mathtext renderer, not a full LaTeX engine.

Example: Custom Tick Labels

Plot $\sin(\omega)$ with ticks expressed as multiples of π:

```
w = np.linspace(0, 2*np.pi, 1000)
y = np.sin(w)
ticks = [0, 0.5*np.pi, np.pi, 1.5*np.pi, 2*np.pi]
labels = ["0", r"$0.5\pi$", r"$\pi$", r"$1.5\pi$", r"$2\pi$"]

plt.plot(w, y)
plt.xticks(ticks, labels)
plt.grid(True)
plt.show()
```

Multiple Curves on One Graph

Call `plt.plot` repeatedly before `plt.show()`; each call gets its own style and label.

```
x = np.arange(-10, 10.5, 0.5)
f = 3*x**4 + 2*x**3 + 7*x + 9
g = 5*x**3 + 9*x + 2

plt.plot(x, f, '-xm', label='f(x)')
plt.plot(x, g, '--ok', label='g(x)')
plt.legend()
plt.show()
```

Result: two curves on shared axes, one magenta with x-markers, one black dashed with circles — their intersection points are visible directly.

© Muním Zabidi

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. **Subplots**
5. Specialized Plots
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. Summary

The subplot Function

An array of plots within one figure — each is a subplot:

```
plt.subplot(m, n, p)
```

- *m*, *n*: grid size (rows, columns)
- *p*: position, numbered left to right, then top to bottom — the opposite convention to array indexing

Example: Stacked Subplots

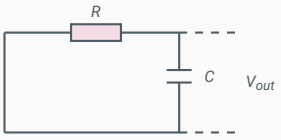
```
x = np.linspace(0, 10, 100)
y = np.cos(3 * x)
z = np.sin(x)

plt.subplot(2, 1, 1)
plt.plot(x, y)
plt.title('y = cos(3x)')

plt.subplot(2, 1, 2)
plt.plot(x, z)
plt.title('z = sin(x)')

plt.tight_layout()   # adjusts subplots to avoid overlap
plt.show()
```

R and C form a voltage divider; the output is taken across the capacitor.



$$H = \frac{1}{1 + j2\pi fRC}$$

$$|H| = \frac{1}{\sqrt{1 + (2\pi fRC)^2}}, \quad \angle H = -\tan^{-1}(2\pi fRC)$$

Since capacitor impedance falls as frequency rises, less input reaches the output at high frequency — a low-pass filter.

© Munim Zabidi

Magnitude and Phase of the RC Filter

```
C, R = 0.47e-6, 4.7e3
f = np.linspace(0, 10000, 10000)

H = 1 / (1 + 1j * 2 * np.pi * f * R * C)
Magnitude = np.abs(H)
Phase = np.angle(H)

plt.subplot(2, 1, 1)
plt.plot(f / 1000, Magnitude)
plt.xlabel('Frequency (kHz)')

plt.subplot(2, 1, 2)
plt.plot(f / 1000, Phase / np.pi)
plt.ylabel(r'Phase ($\pi$ rad)')
```

`np.abs(H)` and `np.angle(H)` give magnitude and phase automatically from a complex transfer function.

Quadrant Subplots: A 2-by-2 Grid

Plot four sinusoids of different frequencies in one figure:

```
t = np.arange(0, 1.001, 0.001)
for F in range(1, 5):
    y = np.sin(2 * np.pi * F * t)
    plt.subplot(2, 2, F)
    plt.plot(t, y)
    plt.title(f'{F} Cycle')

plt.tight_layout()
plt.show()
```

Result: four small plots in a 2×2 grid, each showing one more oscillation than the last.

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. **Specialized Plots**
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. Summary

Stem Plot: Discrete Signals

`plt.stem` draws a marker at each sample connected to a baseline — for discrete-time signals, not continuous ones:

```
n = np.arange(0, 101)
w = 0.04 * np.pi
y = np.sin(w * n)

plt.stem(n, y)
plt.xlabel('sample, n')
plt.ylabel(r'sin($\omega n$)')
plt.show()
```

Each of the 101 samples is drawn as a separate marker on a vertical stem, not joined by a continuous line.

Logarithmic Scales

A nonlinear scale for data spanning a large range of magnitudes:

- Earthquake strength (Richter), sound loudness (decibel), pH, frequency range (Bode plot)

semilogx	semilogy	loglog
x-axis log	y-axis log	both axes log

Example: Loudness, Linear vs. Log

Loudness spans library ($\sim 10^2$ W) to a rock concert ($\sim 10^{11}$ W):

```
L = [7.94e2, 1.26e6, 3.16e8, 1.58e9, 6.3e9, 1.99e11]

plt.subplot(2, 1, 1)
plt.plot(L, '-bo')          # linear: small values near zero, invisible
plt.subplot(2, 1, 2)
plt.semilogy(L, '-bo')      # log: all six values spread out clearly
```

Why it matters

On the linear plot, the four smallest values are visually indistinguishable near zero. The log plot reveals detail across the entire range.

A Bode plot shows how a system responds across frequency — the standard tool for filters, amplifiers, control systems.

Magnitude

How much the output is scaled, in decibels: $20 \log_{10} |H|$.

Phase

How far the output is shifted in time, in degrees.

Two stacked plots sharing one logarithmic frequency axis.

© Muním Zabidi

Generating the Frequency Axis: logspace, Not linspace

Because the frequency axis is logarithmic, frequencies must be generated with `np.logspace`:

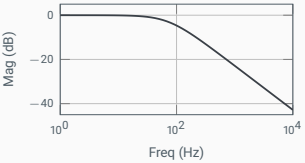
```
R, C = 4.7e3, 0.47e-6
fc = 1 / (2 * np.pi * R * C)   # cutoff, about 72 Hz

f = np.logspace(0, 4, 500)     # 1 Hz to 10 kHz, log-spaced
H = 1 / (1 + 1j * 2 * np.pi * f * R * C)
magnitude_db = 20 * np.log10(np.abs(H))
phase_deg = np.degrees(np.angle(H))
```

Why it matters

A linear range of 10,000 points would place only 9 below 10 Hz and crowd 9,000 above 1 kHz — barely sampling the bend, the part worth seeing.

Reading the Bode Plot



- **Flat passband.** Below ~ 20 Hz, magnitude sits at 0 dB.
- **Cutoff** $f_c \approx 72$ Hz: magnitude falls to -3 dB, phase is $-45°$.
- **Roll-off.** Above cutoff, -20 dB per decade.

This is exactly how unwanted high-frequency noise is removed from a measured signal.

Table of Contents

1 Why Visualize Data?

2 2-D Line Plots

3 Plot Formatting

4 Subplots

5 Specialized Plots

6 **Statistical Plots**

7 The Bench Experiment

8 Preview: Slope

9 Summary

Histogram: Shape of a Dataset

```
data = np.random.randn(1000)
plt.hist(data, bins=50)
plt.xlabel('Data Value')
plt.ylabel('Frequency')
plt.show()
```

What a histogram reveals

Central tendency, spread, and shape of a dataset — useful for spotting outliers, a standard tool in Exploratory Data Analysis.

Box Plots: Median, Quartiles, Outliers

```
plt.boxplot(data)
plt.ylabel('Value')
plt.show()
```

What the box shows

The box spans the interquartile range (25th to 75th percentile), the line inside is the median, and points beyond the whiskers are flagged as potential outliers.

Variants exist for grouped, notched, and horizontal comparisons — the IQR rule behind this returns in Chapter 11.

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. Specialized Plots
6. Statistical Plots
7. **The Bench Experiment**
8. Preview: Slope
9. Summary

101 rows — far too many to read as numbers, exactly right to see at a glance as a picture:

```
import pandas as pd
data = pd.read_csv('bench_rc.csv')
t = data['time_s']
v = data['voltage_V']

plt.plot(t, v, 'o-', markersize=3, label='Measured $v(t)$')
plt.xlabel('Time (s)')
plt.ylabel('Capacitor voltage (V)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```

© Muním Zabidi

A plot should always be checked before you trust it:

- **Units.** Axis runs 0–12 V, matching the 12 V supply.
- **Magnitude.** Starts at 12 V, decays toward zero — as a discharging capacitor must.
- **Limiting case.** At $t = \tau = 0.47$ s, voltage should be $e^{-1} \approx 37\%$ of 12 V ≈ 4.4 V. The graph agrees.
- **Independent check.** $i = v/R$ at $t = 0$ gives 2.55 mA, matching the recorded current.

What the plot reveals that numbers would not

A visible break near $t = 0.8$ s — three dropped samples. Chapter 11 shows how to fix this kind of gap.

© Muním Zabidi

Table of Contents

1 Why Visualize Data?

2 2-D Line Plots

3 Plot Formatting

4 Subplots

5 Specialized Plots

6 Statistical Plots

7 The Bench Experiment

8 **Preview: Slope**

9 Summary

Chapter 6 previewed accumulation. Plotting makes its counterpart just as visible: the slope — different at every point along a curve.

$$\text{slope} = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

For $y = x^2$: gentle rise between $x = 0$ and $x = 1$, steep rise between $x = 3$ and $x = 4$ — same curve, very different local slope.

© Munim Zabidi

Measuring Slope Over Two Intervals

```
x = np.linspace(0, 4, 100)
y = x**2

for x1, x2 in [(0, 1), (3, 4)]:
    slope = (x2**2 - x1**2) / (x2 - x1)
    print(f"Slope {x1} to {x2}: {slope}")
```

```
Slope 0 to 1: 1.0
Slope 3 to 4: 7.0
```

Slope is almost always a rate

Charge vs. time → slope is current. Position vs. time → slope is velocity. Shrinking the interval to a point is exactly what a derivative does (Chapter 13).

Table of Contents

1. Why Visualize Data?
2. 2-D Line Plots
3. Plot Formatting
4. Subplots
5. Specialized Plots
6. Statistical Plots
7. The Bench Experiment
8. Preview: Slope
9. **Summary**

What You Should Take Away

- `plt.plot()` draws a 2-D line plot; it needs axis labels with **units**, a title, and a legend for multiple curves.
- `plt.subplot()` arranges several plots in a grid — how related quantities are compared on a common axis.
- **Logarithmic axes** turn exponential and power-law relationships into straight lines.
- A **Bode plot** shows magnitude (dB) and phase against log frequency; generate frequencies with `np.logspace`, not `np.linspace`.

© Muním Zabidi

What You Should Take Away (continued)

- **Histograms** and **box plots** reveal a dataset's shape, spread, and outliers.
- Always run **engineering sanity checks** on a plot: units, magnitude, limiting cases, independent checks.
- The **slope** of a curve is a rate — shrinking the interval to a point gives the derivative.

Next: Complex Numbers and AC Circuits

© Muním Zabidi